

CHAPTER

Development Environment & Package Management

Mastering pip, Virtual Environments, Dependency Management, and Project Configuration

Learning Objectives

By the end of this chapter, you will be able to:

- Install, upgrade, and uninstall Python packages using pip.
- Understand and apply version specifiers and requirement specifiers correctly.
- Manage package dependencies for both applications and distributable libraries.
- Install packages from PyPI, GitHub, local paths, and wheel files.
- Configure pip using configuration files for different scopes.
- Manage and document project dependencies using requirements.txt and pyproject.toml.
- Resolve common dependency conflicts effectively.

THEORETICAL FOUNDATIONS

1.1 Introduction to Python Package Management

In modern software development — and especially in the fields of Data Science and Generative AI — a single project rarely depends on the Python standard library alone. Developers routinely rely on dozens of third-party libraries such as `numpy`, `pandas`, `torch`, and `transformers`. As a project grows in complexity, managing these libraries — tracking their versions, handling their sub-dependencies, and ensuring environment reproducibility — becomes one of the most critical and challenging aspects of professional development.

Python's answer to this challenge is `pip` — the Package Installer for Python. `pip` connects your local development environment to PyPI (the Python Package Index), a vast public repository hosting hundreds of thousands of open-source Python packages. Understanding `pip` at a deep level is a foundational skill for any Python practitioner.

Key Term: PyPI

PyPI (Python Package Index), accessible at <https://pypi.org>, is the official third-party software repository for Python. It is the default source from which `pip` downloads packages. As of 2024, PyPI hosts over 500,000 packages maintained by developers worldwide.

1.2 Understanding pip — The Package Installer for Python

`pip` is the standard package manager for Python and is distributed with Python versions 3.4 and above. The canonical way to invoke `pip` is through the `python -m pip` syntax. The `-m` flag tells Python to execute the `pip` module as a script, which ensures that the `pip` being invoked corresponds exactly to the currently active Python interpreter — a critical distinction when multiple Python versions or virtual environments coexist on a single machine.

Important: Why `python -m pip` Instead of Just `pip`?

Using `python -m pip` rather than just `pip` guarantees that packages are installed into the intended Python environment. On systems with multiple Python versions (e.g., Python 3.9 and 3.11 co-installed), a bare `pip` may refer to a different interpreter than `python`, causing confusing installation mismatches. Always prefer the explicit form.

1.2.1 Core pip Commands

The following table presents the most commonly used `pip` commands that every Python developer must know:

Command	Description & Purpose
<code>python -m pip install <pkg></code>	Installs the latest stable release of a package from PyPI.
<code>python -m pip install "<pkg>==x.y.z"</code>	Installs a specific, pinned version of a package.
<code>python -m pip install "<pkg>>=x.y"</code>	Installs the package at version x.y or any newer version.

<code>python -m pip install --upgrade <pkg></code>	Upgrades an already-installed package to its latest version.
<code>python -m pip uninstall <pkg></code>	Removes a package from the current Python environment.
<code>python -m pip show <pkg></code>	Displays package metadata: version, author, license, installation path, and dependencies.
<code>python -m pip list</code>	Lists all packages currently installed in the active environment, along with their versions.
<code>python -m pip download <pkg></code>	Downloads the package archive to disk without installing it — useful for offline deployment.
<code>python -m pip freeze</code>	Outputs all installed packages and pinned versions in requirements.txt format.

1.3 Requirement Specifiers and Version Pinning

A **requirement specifier** is the formal syntax used to specify a package and optionally constrain its version during installation. According to **PEP 508** — the Python Enhancement Proposal governing dependency specification syntax — a requirement specifier is composed of a project name followed by optional version specifiers.

The general syntax is: `PackageName[extras] VersionSpecifier; EnvironmentMarker`

Enclosing the version constraint in quotes (e.g., `"arrow>=1.2.0"`) is a best practice when using operators such as `>`, `<`, `>=`, `<=`, or `!=`. Without quotes, certain shell environments may interpret these characters as I/O redirection operators, causing unexpected errors.

1.3.1 Version Specifier Operators

Python's versioning system, defined in PEP 440, supports the following comparison operators for version specifiers:

Operator	Meaning and Example
<code>==</code> (Version Matching)	Exact match only. <code>arrow==1.2.3</code> installs precisely version 1.2.3.
<code>!=</code> (Version Exclusion)	Excludes a specific version. <code>arrow!=1.2.1</code> installs any version except 1.2.1.
<code>>=</code> (Inclusive Lower Bound)	Installs the specified version or any newer. <code>arrow>=1.2</code> .
<code><=</code> (Inclusive Upper Bound)	Installs the specified version or any older. <code>arrow<=2.0</code> .

> (Exclusive Lower Bound)	Installs any version strictly newer than specified. <code>arrow>1.2</code> .
< (Exclusive Upper Bound)	Installs any version strictly older than specified. <code>arrow<2.0</code> .
~= (Compatible Release)	Installs the specified version or newer, but only within the same major/minor release. <code>requests~=2.24.0</code> allows 2.24.x but not 2.25.
=== (Arbitrary Equality)	String-equality match; rarely used. Bypasses normalization rules.

Note: Combining Multiple Specifiers

Multiple version specifiers can be combined with commas, which act as a logical AND operator. For example: `"requests>=2.24, ==2.*"` requires a version that is at least 2.24 AND belongs to the 2.x family — effectively constraining to 2.24 through 2.999.

1.3.2 Environment Markers

Environment markers allow conditional installation based on the runtime environment — such as the Python version, operating system, or platform. This is especially useful in cross-platform projects.

Marker Example	Condition
<code>asyncio;</code> <code>python_version=='3.4'</code>	Install only when Python version is exactly 3.4
<code>typing;</code> <code>python_version<'3.5'</code>	Install only when Python version is below 3.5
<code>asyncio; os_name=='posix'</code>	Install on Mac, Linux, and UNIX systems
<code>asyncio; os_name=='nt'</code>	Install on Windows systems
<code>asyncio;</code> <code>sys_platform=='darwin'</code>	Install on macOS (Darwin kernel) only

1.4 Understanding Package Dependencies

In Python packaging, a **dependency** is any external package that a given package requires to function correctly. When pip installs a package, it also automatically resolves and installs all of that package's listed dependencies. This cascade can be deep: dependencies may themselves have their own dependencies, creating a **dependency tree**.

For example, installing `fastapi` triggers the automatic installation of several packages, including `starlette`, `pydantic`, `anyio`, `idna`, `sniffio`, and `typing-extensions`. Each of these may pull in further sub-dependencies.

1.4.1 Optional (Extra) Dependencies

Some packages define **optional dependencies**, also known as **extras**. These are additional features or integrations that are not required for the core functionality of the package but provide extended capabilities. Unlike regular dependencies, extras are **not installed automatically** — they must be explicitly requested by the user.

The syntax to install extras uses square brackets: `pip install "fastapi[doc,dev]"`. Multiple extras can be specified as a comma-separated list inside the brackets.

Reference: PEP 508 and PEP 440

PEP 508 defines the full specification for dependency requirement strings, covering project names, version constraints, extras, and environment markers.

PEP 440 defines the version identification scheme used by Python packages — how version numbers are structured, ordered, and compared. Both documents are essential references for any Python developer working with package distribution.

1.5 Installing Packages from Various Sources

While PyPI is the default and most common source for package installation, pip supports a variety of installation sources to accommodate real-world development scenarios.

1.5.1 Installing from GitHub (Version Control Systems)

Not every useful Python package is published on PyPI. Research code, internal tools, and bleeding-edge library builds are frequently distributed exclusively via GitHub or other version control systems. pip supports installing directly from a VCS URL using the `git+` prefix.

The general syntax for installing from a GitHub repository is:

```
# Install from the default branch of a GitHub repository
```

```
python -m pip install "PackageName @
git+https://github.com/username/repo.git"
```

```
# Install from a specific branch
```

```
python -m pip install "PackageName @
git+https://github.com/username/repo.git@main"
```

```
# Install from a specific commit hash
```

```
python -m pip install "PackageName @
git+https://github.com/username/repo.git@da39b"
```

```
# Install from a version tag
```

```
python -m pip install "PackageName @
git+https://github.com/username/repo.git@v2.1"
```

Tip: VCS Support Documentation

pip's documentation includes a dedicated **VCS Support** section that describes the full syntax for installing from Git, Mercurial, Subversion, and Bazaar repositories. It is recommended reading for developers who work with private or pre-release packages.

1.5.2 Installing from the Local File System

When a package is available as a local directory (containing a `pyproject.toml` or `setup.py`) or as a pre-built wheel file (`.whl`), pip can install directly from the local path.

```
# Install a local package from its directory path
```

```
python -m pip install ~/projects/demo_pkg/
```

```
# Install a pre-built wheel file
```

```
python -m pip install myproject-1.1-py3-none-any.whl
```

```
# Editable install (development mode) – no file copying
```

```
pip install -e ~/projects/demo_pkg
```

The `-e` flag (short for `--editable`) installs the package in **editable mode**. In this mode, the package is linked directly from its source directory rather than being copied into the site-packages folder. Any changes made to the source code are immediately reflected in the installed package — without requiring a reinstall. This is the preferred mode when actively developing a package.

1.5.3 The --user Flag

On systems where the user does not have administrative privileges, the `--user` flag instructs pip to install packages into a user-specific directory rather than the system-wide `site-packages`. This is often found at `~/.local/lib/pythonX.Y/site-packages` on Linux and macOS.

```
# Install a package for the current user only (no admin rights needed)
python -m pip install --user requests
```

Note: Linux Built-in Python Environment

On some Linux distributions, Python is provided as a built-in system utility. The system-managed Python environment is intentionally isolated from user-installed packages to prevent conflicts with system tools. On Windows and macOS, Python is not bundled with the OS — it must be installed by the user, and no such isolation concern exists.

1.6 Configuring pip with Configuration Files

When working in professional environments, it is common to have non-default pip settings that apply consistently — such as using a private package repository (e.g., a corporate Artifactory or Nexus instance), routing traffic through a proxy server, or trusting a custom SSL certificate. Specifying these settings on every single pip command would be impractical and error-prone.

pip supports a **configuration file** mechanism that allows these options to be defined once and applied automatically to all subsequent pip commands. Configuration files are written in INI format.

1.6.1 Configuration Scopes

pip recognizes three configuration scopes, applied in order of precedence from lowest to highest:

Scope	Description and Typical File Location
global	System-wide configuration, shared across all users on the machine. On Linux/macOS: <code>/etc/pip.conf</code> . On Windows: <code>C:\ProgramData\pip\pip.ini</code> .
user	Per-user configuration, applying only to the current user account. On Linux/macOS: <code>~/.config/pip/pip.conf</code> . On Windows: <code>%APPDATA%\pip\pip.ini</code> .
site	Per-virtual-environment configuration. Placed inside the active virtual environment directory. Takes highest precedence.

1.6.2 Common Configuration File Use Cases

The following are the most common scenarios for which a pip configuration file is created:

- **Hosting a private package repository** — redirect pip to use an internal Artifactory, Nexus, or DevPI server instead of the public PyPI.
- **Using a proxy server** — route all pip network requests through a corporate proxy.
- **Custom SSL certificates** — specify a custom certificate authority bundle for environments with self-signed certificates.
- **Setting a default index URL** — define a primary or extra package index without specifying it on every command.

1.7 Managing Project Dependencies

A fundamental distinction exists in Python packaging between two categories of projects: applications that you deploy and run yourself, and libraries that you distribute to other developers. Each category demands a different strategy for managing dependencies.

1.7.1 Applications — requirements.txt

An **application** is a Python project that you deploy and operate yourself — a web service, a data pipeline, a machine learning training script, or a batch processing job. For applications, the primary dependency management tool is the **requirements file**, conventionally named `requirements.txt`.

A requirements file is a plain text file, each line of which contains a requirement specifier. pip reads this file using the `-r` flag to install all listed dependencies in one command:

```
# Install all dependencies from a requirements file
python -m pip install -r requirements.txt
```

In large projects, it is common practice to maintain multiple requirements files for different purposes:

- `requirements.txt` — core runtime dependencies for production.
- `requirements-dev.txt` — additional tools needed only during development (e.g., pytest, black, mypy).
- `requirements-test.txt` — dependencies required for running automated tests.

Best Practice: Pin Versions in requirements.txt

For applications, it is best practice to **pin all dependency versions** (e.g., `numpy==1.26.4`) in your requirements file. Version pinning guarantees that your deployed environment is identical to your development and testing environment, eliminating the notorious 'works on my machine' problem. Use `pip freeze > requirements.txt` to automatically capture the exact current state of your environment.

1.7.2 Libraries — pyproject.toml

A **library** is a Python package intended to be distributed and used by other developers — whether published to PyPI, shared internally within a company, or distributed as part of an open-source project. Libraries use the **pyproject.toml** file as the modern standard for defining project metadata, build configuration, and dependencies.

A key distinction when writing a library is that you should **not pin dependency versions**. Pinning creates rigid constraints that make it very difficult for library users to install your package alongside their own dependencies. Instead, specify version ranges — e.g., `"numpy>=1.20"` — to express the minimum requirements while allowing flexibility.

Key Concept: Backtracking

When pip encounters conflicting version requirements between packages, it uses a process called **backtracking**. pip will attempt multiple combinations of package versions until it finds a set that satisfies all constraints simultaneously. If no such combination exists, pip reports a dependency conflict error. This is why overly strict version pinning in library dependencies is harmful — it dramatically increases the probability of irresolvable conflicts.

1.7.3 Handling Dependency Conflicts

Dependency conflicts arise when two or more packages in the same environment require mutually incompatible versions of a shared dependency. Common resolution strategies include:

1. **Relaxing version constraints** — if you control one of the conflicting packages, loosen its version specifiers to allow overlap.
2. **Upgrading or downgrading a dependency** — manually select a version of the conflicting package that satisfies all requiring parties.
3. **Using virtual environments** — isolate conflicting projects in separate virtual environments so they do not interfere with one another.
4. **Replacing a dependency** — if a dependency is a persistent source of conflicts, consider whether an alternative library could meet the same need.

1.8 Virtual Environments and the Development Workflow

A **virtual environment** (venv) is an isolated Python environment that contains its own Python interpreter, pip installation, and site-packages directory. Virtual environments are the cornerstone of professional Python project management — they prevent package version conflicts between different projects and keep the global Python installation clean.

```
# Create a virtual environment named 'venv' in the current directory
python -m venv venv

# Activate the virtual environment
# On Linux/macOS:
source venv/bin/activate

# On Windows (Command Prompt):
venv\Scripts\activate.bat

# On Windows (PowerShell):
venv\Scripts\Activate.ps1

# Once activated, pip commands apply only to this environment
pip install fastapi

# Deactivate when done
deactivate
```

Integration with VS Code

When using Visual Studio Code, you must manually create a virtual environment and then select it as the Python interpreter for your workspace. In VS Code, press **Ctrl+Shift+P** (or **Cmd+Shift+P** on macOS), search for **Python: Select Interpreter**, and choose the interpreter inside your **venv** folder. All terminal operations in VS Code will then automatically use the selected virtual environment.

PRACTICAL & CODING REFERENCE

This section provides a comprehensive, cell-by-cell walkthrough of all pip and package management commands. Each cell includes the command itself, a conceptual explanation of what it does, the rationale for using this approach, and a description of the expected output.

Cell 1 — Installing a Package from PyPI

```
python -m pip install arrow
```

Concept

arrow is a Python library for working with dates and times in a more human-friendly way than the standard `datetime` module. It provides intelligent creation, manipulation, formatting, and conversion of date-time objects.

Why This Approach?

The `python -m pip install` syntax is the preferred invocation over a bare `pip install`. It explicitly ties the pip operation to the active Python interpreter, eliminating ambiguity in multi-version environments. When inside a virtual environment, this command installs the package exclusively into that environment.

Expected Output

pip will first check if arrow is already installed. If not, it connects to PyPI, resolves the latest stable version, downloads the package archive (and any dependencies), and installs them. A typical output ends with:

```
Successfully installed arrow-1.3.0
```

Cell 2 — Inspecting a Package's Metadata

```
python -m pip show arrow
```

Concept

The **show** command queries the installed package database and displays metadata for the specified package. This metadata is read from the package's `METADATA` file located in the site-packages directory.

Why This Approach?

This is the fastest way to confirm a package's installed version, check its physical installation path on disk, view its license and author information, and inspect which other packages it depends on (**Requires**) and which packages depend on it (**Required-by**). Essential for debugging dependency issues.

Expected Output

The output is a structured key-value display similar to:

```
Name: arrow
Version: 1.3.0
Summary: Better dates & times for Python
Home-page: https://arrow.readthedocs.io
Location: /path/to/venv/lib/python3.11/site-packages
Requires: python-dateutil
Required-by:
```

Cell 3 — Listing All Installed Packages

```
python -m pip list
```

Concept

The **list** command enumerates every package installed in the currently active Python environment, alongside its installed version number.

Why This Approach?

Use **pip list** to get an overview of the full environment state. This is useful when diagnosing unexpected behaviour (a wrong version may be installed), when auditing an environment before deployment, or when onboarding to a new project. For capturing the state in a format suitable for a requirements file, prefer **pip freeze** instead.

Expected Output

A formatted table of all installed packages:

Package	Version
-----	-----
arrow	1.3.0
pip	24.0
python-dateutil	2.9.0
setuptools	69.0.3

Cell 4 — Installing a Specific Version

```
# Exact version pin
python -m pip install "arrow==1.2.3"

# Minimum version requirement
python -m pip install "arrow>=1.2"

# Exclusive upper bound
python -m pip install "arrow<2.0"

# Exclude a specific problematic version
python -m pip install "arrow!=1.2.1"
```

Concept

Version specifiers allow precise control over which version of a package pip selects. The specifier is appended to the package name using version operators defined in **PEP 440**. Quotation marks are required when using comparison operators to prevent shell interpretation.

Why This Approach?

Pinning exact versions (`==`) ensures complete reproducibility across environments — critical for production deployments. Range specifiers (`>=` , `<`) are preferred in library `pyproject.toml` files to allow users flexibility. Exclusion specifiers (`!=`) are useful when a specific version is known to contain a bug or breaking change.

Expected Output

Each command outputs a success message indicating the version selected and installed. For `arrow==1.2.3` : **Successfully installed arrow-1.2.3 python-dateutil-2.8.2**

Cell 5 — Compatible Release Specifier

```
# Compatible release: same major.minor, any patch version
python -m pip install "requests~=2.24.0"

# Equivalent to: requests>=2.24.0, ==2.24.*
```

```
python -m pip install "requests>=2.24, ==2.*"
```

Concept

The compatible release operator `~=` is a convenient shorthand for 'at least this version, but staying within the same release family.' For `~=2.24.0`, pip will install 2.24.0 or higher, but never 2.25 or beyond. This aligns with semantic versioning expectations — patch releases are safe to upgrade, but minor releases may introduce breaking changes.

Why This Approach?

The compatible release specifier is ideal for libraries that follow semantic versioning. It provides a balance between reproducibility (staying within a tested minor version) and flexibility (allowing bug-fix patches to be applied automatically).

Cell 6 — Installing a Package with Optional Extras

```
# Install FastAPI with optional documentation and development extras
python -m pip install "fastapi[doc,dev]"
```

```
# Install multiple extras for a data science package
python -m pip install "pandas[performance,plot]"
```

Concept

Extras are optional feature bundles defined by the package author. They group together additional dependencies needed for a specific use case. The extras for a package are documented in its `pyproject.toml` under `[project.optional-dependencies]` and are visible on the package's PyPI page.

Why This Approach?

Extras promote a lean core installation. Users who don't need documentation-building tools or development utilities won't have those packages cluttering their environment. This is particularly valuable in production containers where image size matters. Only install the extras you actually need.

Expected Output

Installing `fastapi[doc,dev]` installs FastAPI's core dependencies plus the extra packages specified for the 'doc' and 'dev' extras, such as `httpx`, `pre-commit`, and documentation generators.

Cell 7 — Installing Packages from GitHub

```
# Install directly from the default branch of a GitHub repository

python -m pip install "demo_pkg @ git+https://github.com/codesensei-
courses/demo_pkg.git"
```

```
# Install from a specific branch

python -m pip install "demo_pkg @
git+https://github.com/user/repo.git@main"
```

```
# Install from a specific version tag

python -m pip install "demo_pkg @
git+https://github.com/user/repo.git@v2.1.0"
```

Concept

pip's VCS (Version Control System) support allows direct installation from any public or accessible private Git repository. This is essential for using forks of packages, pre-release code that hasn't been published to PyPI yet, or internal proprietary packages hosted on private repositories. The **git+** prefix signals to pip that this is a VCS URL rather than a package name.

Why This Approach?

In Data Science and GenAI projects, you will frequently encounter cutting-edge models, experimental libraries, and research code that is available on GitHub but not on PyPI. The ability to install directly from source is indispensable. For reproducibility, always pin to a specific commit hash or version tag rather than a branch name (branches are mutable — the 'main' branch today is not the same as 'main' in three months).

Cell 8 — Installing from Local Path and Editable Mode

```
# Navigate to the project directory

cd ~/projects/demo_pkg

# Install the local package

python -m pip install ~/projects/demo_pkg/
```

```
# Install in editable mode (for active development)
```

```
pip install -e ~/projects/demo_pkg
```

```
# Verify the editable install is registered
```

```
python -m pip show demo_pkg
```

Concept

A standard local install copies the package files into site-packages. An **editable install** (`-e` or `--editable`) creates a special `.pth` file or `direct_url.json` pointer that makes Python look at the original source directory. There is no file copying — your live source code IS the installed package.

Why This Approach?

Editable installs are the standard workflow for package development. Without them, you would need to re-run pip install after every code change to see the effect. With editable mode, you edit the file, switch to your REPL or test suite, and the change is immediately available — dramatically accelerating the development feedback loop.

Cell 9 — Downloading Packages Without Installing

```
# Download the package archive(s) to the current directory
```

```
python -m pip download requests
```

```
# Download to a specific directory
```

```
python -m pip download requests -d ./offline_packages/
```

```
# Download a specific version
```

```
python -m pip download "requests==2.31.0" -d ./offline_packages/
```

Concept

The `pip download` command fetches package archives (wheel files or source distributions) and saves them to disk without installing them. All dependencies are downloaded as well by default.

Why This Approach?

This command is invaluable for offline or air-gapped deployment scenarios — for example, when deploying to a server with no internet access, or when pre-populating a Docker build cache. You download the packages once on an internet-connected machine, transfer the archives, and then install from the local directory on the target machine.

Cell 10 — Working with requirements.txt

```
# Generate a requirements.txt from the current environment
python -m pip freeze > requirements.txt

# contents of requirements.txt (example):
# arrow==1.3.0
# fastapi==0.110.0
# numpy==1.26.4
# pydantic==2.6.3

# Install all packages from a requirements file
python -m pip install -r requirements.txt

# Install from multiple requirements files
python -m pip install -r requirements.txt -r requirements-dev.txt
```

Concept

A `requirements.txt` file is a plain text list of requirement specifiers, one per line. `pip` processes this file line by line when the `-r` flag is provided. Comments (lines starting with `#`) are supported. A requirements file can even reference other requirements files using `-r another_file.txt` within it.

Why This Approach?

The `pip freeze > requirements.txt` workflow captures the exact, complete state of a working environment. When another developer (or a CI/CD pipeline) runs `pip install -r requirements.txt`, they receive an identical environment. This is the single most important practice for ensuring reproducibility in application development.

Cell 11 — A Sample pyproject.toml for Library Distribution

```
[project]
name = "demo_pkg"
version = "0.0.1"
dependencies = [
    "arrow>=1.2.0",
    'importlib-metadata; python_version<"3.8"',
]

[project.optional-dependencies]
dev = ["pytest", "black", "mypy"]
docs = ["sphinx", "sphinx-rtd-theme"]

[project.scripts]
# Entry point: running 'demo' in terminal calls demo_pkg.demo:main
demo = "demo_pkg.demo:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Concept

The `pyproject.toml` file is the modern, unified configuration file for Python projects, standardised by **PEP 517** and **PEP 518**. It replaces the older `setup.py` and `setup.cfg` approach. It defines project metadata (name, version, authors, license), runtime dependencies, optional extras, entry points, and build system configuration all in one file.

Why This Approach?

For a distributable library, `pyproject.toml` is the industry standard. Notice that the dependencies use range specifiers (`>=`) rather than pinned versions (`=`). This is intentional — a library must be flexible enough to coexist with the rest of a user's environment. Pinning to an exact version in a library would force that version on every user, creating widespread dependency conflicts.

Line-by-Line Walkthrough

[project] — The primary metadata section for the project.

name — The unique package name as it will appear on PyPI.

version — Follows semantic versioning (major.minor.patch).

dependencies — List of runtime dependencies with loose version specifiers.

importlib-metadata; python_version < "3.8" — Conditional dependency using an environment marker: only installed if Python < 3.8.

[project.optional-dependencies] — Defines extras groups (dev, docs) that users can opt into.

[project.scripts] — Defines command-line entry points. After installation, running **demo** in the terminal executes the **main()** function in **demo_pkg/demo.py**.

[build-system] — Specifies the build backend (hatchling) used to package the project into a distributable archive.

Cell 12 — Environment Markers in Practice

```
# Install asyncio only on Python 3.4
python -m pip install "asyncio; python_version=='3.4'"

# Install typing only on Python versions below 3.5
python -m pip install "typing; python_version < '3.5'"

# Platform-specific installation
python -m pip install "asyncio; os_name=='posix'" # Linux/macOS/UNIX
python -m pip install "asyncio; os_name=='nt'" # Windows
python -m pip install "asyncio; sys_platform=='darwin'" # macOS only
```

Concept

Environment markers are evaluated at install time and are used to conditionally include a dependency based on the runtime environment. The marker expression — separated from the package name by a semicolon — uses Python-like syntax with access to the following marker variables: **python_version**, **python_full_version**, **os_name**, **sys_platform**, **platform_machine**, and **implementation_name**.

Why This Approach?

Environment markers enable a single dependency specification to be cross-platform and cross-version compatible. Rather than maintaining separate requirements files for Windows and Linux, or for Python 3.7 and 3.11, markers allow all conditions to be expressed in one place. This is essential for professional-grade library distribution.

Quick Reference Summary

The following table provides a consolidated reference for all pip operations covered in this chapter:

Task	Command
Install latest version	<code>python -m pip install arrow</code>
Install exact version	<code>python -m pip install "arrow==1.2.3"</code>
Compatible release	<code>python -m pip install "requests~=2.24.0"</code>
Exclude a version	<code>python -m pip install "arrow!=1.2.1"</code>
Install with extras	<code>python -m pip install "fastapi[doc,dev]"</code>
Upgrade a package	<code>python -m pip install --upgrade arrow</code>
Uninstall a package	<code>python -m pip uninstall arrow</code>
Show package metadata	<code>python -m pip show arrow</code>
List all packages	<code>python -m pip list</code>
Install from GitHub	<code>pip install "pkg @ git+https://github.com/user/repo.git"</code>
Install from local path	<code>python -m pip install ~/projects/my_pkg/</code>
Editable (dev) install	<code>pip install -e ~/projects/my_pkg</code>
Download without install	<code>python -m pip download requests</code>
Freeze environment	<code>python -m pip freeze > requirements.txt</code>
Install from requirements	<code>python -m pip install -r requirements.txt</code>
Install for current user	<code>python -m pip install --user requests</code>

Chapter Summary

In this chapter, we established a thorough understanding of Python's package management ecosystem, which underpins every professional Data Science and GenAI project:

5. **pip and PyPI** — pip is the primary package installer for Python, drawing packages from the Python Package Index. Always invoke it as `python -m pip` to avoid interpreter ambiguity.
6. **Requirement Specifiers** — Version constraints use operators (`==`, `!=`, `>=`, `<=`, `~=`) defined in PEP 440. Always quote specifiers containing comparison operators in shell commands.

7. **Dependencies and Extras** — pip automatically resolves and installs transitive dependencies. Optional extras require explicit specification using square-bracket syntax.
8. **Installation Sources** — Packages can be installed from PyPI, GitHub (VCS URLs), local directories, wheel files, or pre-downloaded archives.
9. **pip Configuration** — pip configuration files (global, user, site scopes) enable persistent custom settings such as private indices, proxies, and SSL certificates.
10. **Dependency Management Strategies** — Applications use requirements.txt with pinned versions for reproducibility. Libraries use pyproject.toml with range specifiers for flexibility.
11. **Virtual Environments** — Always develop inside a virtual environment to isolate project dependencies and maintain clean, reproducible setups.

What's Next?

In the next chapter, we will explore advanced virtual environment management with tools such as conda and poetry, and examine how to structure production-ready Python projects for Data Science and GenAI applications.